

A Technical Introduction to Masked language model

Section 1

Interpretation Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. Their performance on these natural language understanding tasks are not yet well understood. Several research publications in 2018 and 2019 focused on investigating the relationship behind BERT's output as a result of carefully chosen input sequences, analysis of internal vector representations through probing classifiers, and the relationships represented by attention weights. The high performance of the BERT model could also be attributed to the fact that it is bidirectionally trained. This means that BERT, based on the Transformer model architecture, applies its self-attention mechanism to learn information from a text from the left and right side during training, and consequently gains a deep understanding of the context. For example, the word fine can have two different meanings depending on the context (I feel fine today, She has fine blond hair). BERT considers the words surrounding the target word fine from the left and right side. However it comes at a cost: due to encoder-only architecture lacking a decoder, BERT can't be prompted and can't generate text, while bidirectional models in general do not work effectively without the right side, thus being difficult to prompt. As an illustrative example, if one wishes to use BERT to continue a sentence fragment "Today, I went to", then naively one would mask out all the tokens as "Today, I went to [MASK] [MASK] [MASK] ... [MASK] ." where the number of [MASK] is the length of the sentence one wishes to extend to. However, this constitutes a dataset shift, as during training, BERT has never seen sentences with that many tokens masked out. Consequently, its performance degrades. More sophisticated techniques allow text generation, but at a high computational cost.

Section 2

History BERT was originally published by Google researchers Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. The design has its origins from pre-training contextual representations, including semi-supervised sequence learning, generative pre-training, ELMo, and ULMFit. Unlike previous models, BERT is a deeply bidirectional, unsupervised language representation, pre-trained using only a plain text corpus. Context-free models such as word2vec or GloVe generate a single word embedding representation for each word in the vocabulary, whereas BERT takes into account the context for each occurrence of a given word. For instance, whereas the vector for "running" will have the same

word2vec vector representation for both of its occurrences in the sentences "He is running a company" and "He is running a marathon", BERT will provide a contextualized embedding that will be different according to the sentence. On October 25, 2019, Google announced that they had started applying BERT models to English-language search queries on Google Search within the US. On December 9, 2019, it was reported that BERT had been adopted by Google Search for over 70 languages. In October 2020, almost every single English-based query was processed by a BERT model.

Section 3

Cost BERT was trained on the BookCorpus (800M words) and a filtered version of English Wikipedia (2,500M words) without lists, tables, and headers. Training BERTBASE on 4 cloud TPU (16 TPU chips total) took 4 days, at an estimated cost of 500 USD. Training BERTLARGE on 16 cloud TPU (64 TPU chips total) took 4 days.

Section 4

Bidirectional encoder representations from transformers (BERT) is a language model introduced in October 2018 by researchers at Google. It learns to represent text as a sequence of vectors using self-supervised learning. It uses the encoder-only transformer architecture. BERT dramatically improved the state of the art for large language models. As of 2020, BERT is a ubiquitous baseline in natural language processing (NLP) experiments. BERT is trained by masked token prediction and next sentence prediction. With this training, BERT learns contextual, latent representations of tokens in their context, similar to ELMo and GPT-2. It found applications for many natural language processing tasks, such as coreference resolution and polysemy resolution. It improved on ELMo and spawned the study of "BERTology", which attempts to interpret what is learned by BERT. BERT was originally implemented in the English language at two model sizes, BERTBASE (110 million parameters) and BERTLARGE (340 million parameters). Both were trained on the Toronto BookCorpus (800M words) and English Wikipedia (2,500M words). The weights were released on GitHub. On March 11, 2020, 24 smaller models were released, the smallest being BERTTINY with just 4 million parameters.

Section 5

Masked language modeling (MLM): In this task, BERT ingests a sequence of words, where one word may be randomly changed ("masked"), and BERT tries to predict the original words that had been changed. For example, in the sentence "The cat sat on the [MASK]," BERT would need to predict "mat." This helps BERT learn bidirectional context, meaning it understands the relationships between words not just from left to right or right to left but from both

directions at the same time. Next sentence prediction (NSP): In this task, BERT is trained to predict whether one sentence logically follows another. For example, given two sentences, "The cat sat on the mat" and "It was a sunny day", BERT has to decide if the second sentence is a valid continuation of the first one. This helps BERT understand relationships between sentences, which is important for tasks like question answering or document classification.

Section 6

Architectural family The encoder stack of BERT has 2 free parameters: L , the number of layers, and H , the hidden size. There are always $H/64$ self-attention heads, and the feed-forward/filter size is always $4H$. By varying these two numbers, one obtains an entire family of BERT models. For BERT:

Section 7

Given two sentences, the model predicts if they appear sequentially in the training corpus, outputting either [IsNext] or [NotNext]. During training, the algorithm sometimes samples two sentences from a single continuous span in the training corpus, while at other times, it samples two sentences from two discontinuous spans. The first sentence starts with a special token, [CLS] (for "classify"). The two sentences are separated by another special token, [SEP] (for "separate"). After processing the two sentences, the final vector for the [CLS] token is passed to a linear layer for binary classification into [IsNext] and [NotNext]. For example: